Week 4 Notes

Busy Beaver

Jacks discussion of Turning machines, and the discussion of beaver tripples, reminds me of the busy beaver problem. Which is to create the longest terminating program using minimal Turing machine states.

From Busy Beaver wikipedia:

> Deciding the running time or score of the nth busy beaver is noncomputable.

> An implication of the busy beaver game is that, if it were possible to compute the functions Σ(n) and S(n) for all n, then this would resolve all mathematical conjectures that can be reduced to a halting problem, e.g., into a form of "does ⟨this Turing machine⟩ halt".[6] For example, there is a 25-state Turing machine that checks Goldbach's conjecture for each number and halts on a counterexample; if this machine did not halt after running for S(25) steps, then it must run forever, resolving the conjecture.[7] Many other problems, including the Riemann hypothesis (120[8] states) and the consistency of ZF set theory (745 states[9][10]), can be expressed in a similar form, where at most a countably infinite number of cases need to be checked.[6]

I am however pretty sure that this is already well explored, and there are no low hanging fruit for us to try solving.

This is probably also why everyone cares so much about the halting problem.

Avi Widgerson Book

I read the first "Computational Complexity 101" chapter. Avi brings up the important point about the difference of calculating and recognizing the right solution as programming tasks. Verification is much easier, but not necessairly.

I realized that the calculation vs verification difference is important, because it determines our ability to offload work to unreliable 3rd parties.

NP and coNP are about proofs that can be verified in polynomial time. Curiously, I don't think there is a guarantee that even if a result is computable in sub exponential time, a convincing proof can be generated in less time.

It is not intuitive (to me) that problems could be complete in their complexity class. I don't see how every problem in P can be converted into any other in P.

CS Theory Toolkit

Recitation 3

I didn't understand what was happening. Perhaps would be great to have access to that course's homework problem sets.

Lecture

Note: this is heavy overlap with my sta257 material.

A nice thing about chebunchev inequality, is that it works if variables are pairwise independent.

TODO, I didn't understand anything, will have to re-watch all the lectures and work out the math for myself.